什么是 Function Call?

上一章你已经知道了，一个工具 = 函数本体 + 名称 + 描述 + 参数定义，大模型靠工具描述来判断要不要调这个工具。

但这里有个关键问题还没解决：**大模型做出「要调这个工具」的决策之后，它怎么告诉 Agent 去执行?**

大模型只会"说话"，它的输出永远是文字。但 Agent 要调用工具，必须知道：调哪个工具？传什么参数？格式是什么？这中间的信息交换，靠什么来保证准确？

这就是 Function Calling 要解决的核心问题。在讲它是什么之前，我们先来看看没有它的时候，开发者有多惨。

没有 Function Calling 之前：靠自然语言"猜"

在 Function Calling 出现之前，开发者只能把工具的描述和调用规范，全部用**自然语言写进 System Prompt（系统提示词）**里。

比如你想让 AI 能调用一个查天气的工具，System Prompt 里就得这样写：

```
你是一个智能助手，你可以使用以下工具：

工具名：check_weather
功能：查询某个城市的天气
参数：
  - city：城市名，中文，比如"上海"
  - date：日期，格式必须是 YYYY-MM-DD，比如"2026-03-19"
  - date 是可选的，不填默认查今天

当用户需要查天气时，你要调用这个工具，告诉我工具名和参数。
```

当用户需要查天气时，你要调用这个工具，告诉我工具名和参数。

听起来还好？但问题是，大模型是个自由发挥的选手，你约束不住它。你让它查「上海明天的天气」，它可能给你返回：

- **散文式回复**："好的，我来帮你查询上海明天的天气，需要使用 check_weather 工具，城市是上海，时间是明天～"
- **参数顺序颠倒**：`{"date": "明天", "city": "上海"}`（"明天"完全不是 YYYY-MM-DD 格式，直接传进工具就报错）

- **直接编造天气**：不调工具，根据训练数据给你生成一个"明天上海天气预报"
- **JSON 里混自然语言**：`调用工具: check_weather, 参数 city=上海, date=2026-03-20`
- **完全无视要求**：给你一段介绍上海气候特点的科普文章

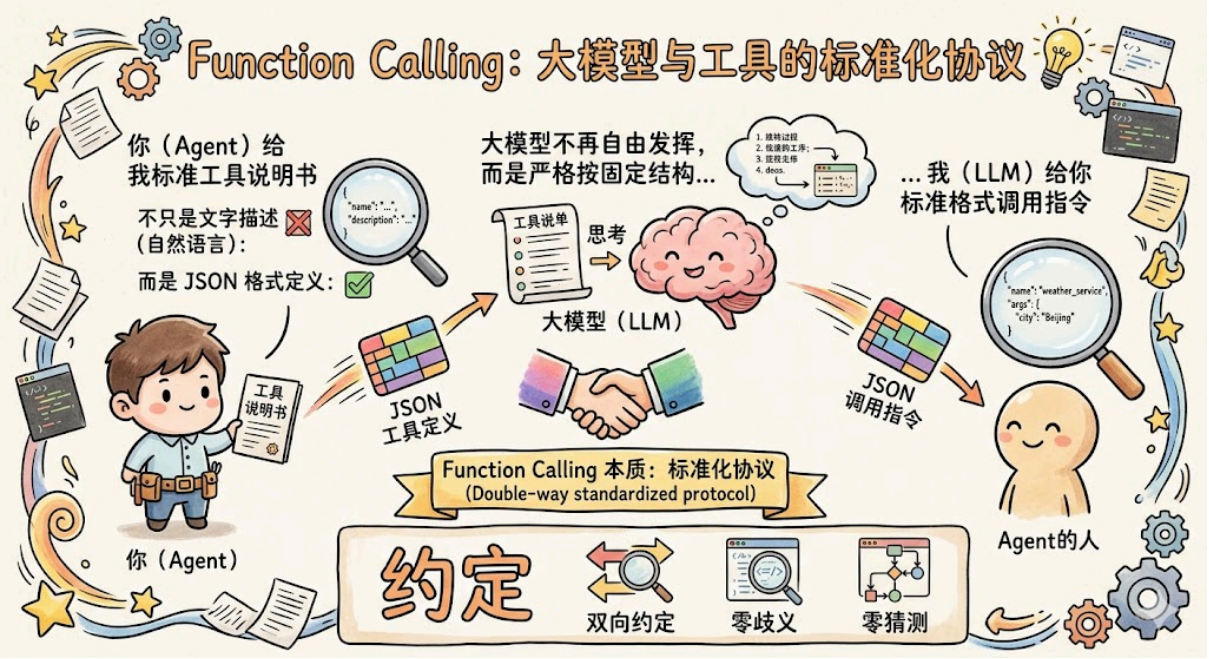
这就是传统 System Prompt 模式的本质问题，**全靠 AI 猜**：猜要不要调工具、猜参数格式、猜回复结构。猜对了正常运行，猜错了整个流程就崩了。

而开发者呢？就得写一大堆正则表达式和解析代码，尝试从 AI 五花八门的回复里"扒"出工具名和参数。工具越多，解析代码比业务代码还长。而且换个模型，还得重写一遍。这种开发体验，完全是噩梦级别的。

Function Calling 的出现：告别猜谜，统一规范

这就是 Function Calling 诞生的背景。

Function Calling 的本质，是一套大模型和 Agent 之间的标准化工具调用协议，我们不再用自然语言告诉 AI「你可以用这个工具」，而是用一套固定的 JSON 格式把工具定义清楚；大模型也不再自由发挥，而是严格按固定的 JSON 结构返回调用指令。



整个过程变成了这样：你给我标准格式的工具说明书，我给你标准格式的调用指令，双向约定，零歧义，零猜测。

这里很多同学会有个疑问：Tool 和 Function Call 有什么关系？简单来说，**Tool 是「能力实体」，Function Call 是「调用机制」**，大模型通过 Function Call 这个协议，去调用 Tool 这个具体的功能。解决的是完全不同层面的问题：

- **工具（Tool）= 能力层，解决「有什么」**：一个工具需要哪些要素才算完整、才能让大模型认识和使用它，函数本体是真正干活的，名称让大模型能精准识别，描述告诉大模型什么时候该用它，参数定义告诉大模型怎么构造调用参数。这是在定义一个能力本身。
- **Function Calling = 协议层，解决「怎么传」**：工具写好之后，大模型和 Agent 之间要用什么格式来交换信息，大模型怎么告诉 Agent 要调哪个工具、传什么参数；Agent 执行完工具后怎么把结果回传给大模型。这是一套标准化通信规范。

打个比方：工具是外卖骑手（有姓名、有技能、能送餐），Function Calling 是美团的派单系统（规定了订单怎么下达、骑手怎么接单、送达后怎么确认）。骑手和派单系统缺一不可，但它们解决的是完全不同层面的两件事，骑手是「能力」，派单系统是「通信规范」。

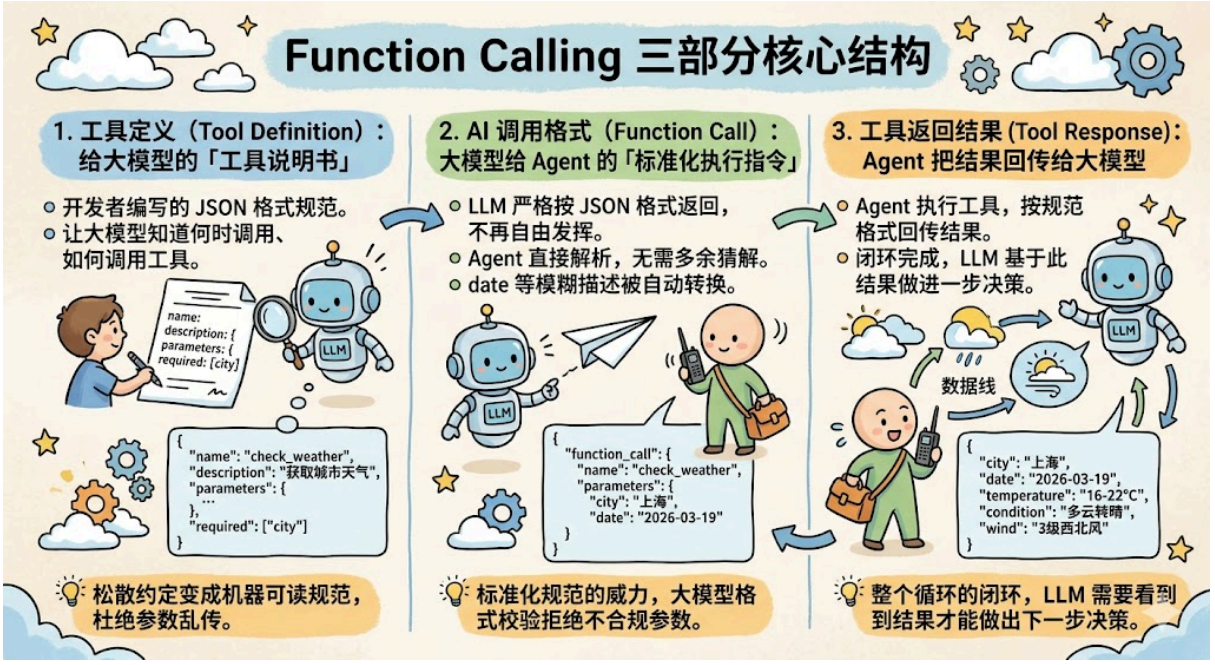
所以Tool章讲的「工具四要素」，是从概念层告诉你一个工具应该有什么；这章讲的 Function Calling，是从协议层告诉你这些要素以什么标准化格式在大模型和 Agent 之间流转。

两者的协作流程，以「查询北京明天天气」为例：

1. 用户提问：「北京明天天气怎么样？」
2. 大模型分析：需要调用天气 Tool，生成 Function Call 格式的指令。
3. 系统解析指令：调用 `get_weather("北京", "2026-03-21")` 这个 Tool。
4. Tool 执行并返回结果：「北京明天晴，10\~20°C」。
5. 大模型把结果整理成自然语言，回复给用户。

Function Calling 三部分核心结构

接下来，我们用天气查询的场景，把 Function Calling 完整的三部分结构拆解清楚，每一行都讲明白作用。



第一部分：工具定义（Tool Definition），给大模型的「工具说明书」

这一步是我们开发者要写的，相当于给大模型一份严格规范的工具说明书，大模型会完全按照这份说明书，判断要不要调用工具、怎么调用工具。

```
{
  "name": "check_weather",
  "description": "获取指定城市指定日期的天气情况，包括温度区间、晴雨状况、风力风向",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "需要查询天气的城市中文名称，示例：北京、上海、广州"
      },
      "date": {
        "type": "string",
        "format": "YYYY-MM-DD",
        "description": "需要查询的日期，格式为年-月-日，示例：2026-03-19，不填默认查询当天"
      }
    },
```

```
      "required": ["city"]
    }
  }
```

这里每个字段都有明确作用：

- `name`：工具的唯一标识，大模型调用时必须和这个名字完全一致
- `description`：工具功能说明，**是大模型判断「要不要调这个工具」的核心依据**，写得越精准，大模型做对决策的概率越高
- `parameters`：告诉大模型这个工具需要哪些参数、什么类型、什么格式，大模型会严格按照这里的约束来生成参数值
- `required`：必填参数清单，如果用户没提供必填信息，大模型会主动去问用户，而不是瞎猜一个

对比之前 System Prompt 里用自然语言描述的"松散约定"，这里的 JSON 格式是严格的机器可读规范，大模型在服务端就会做格式校验，不合规的参数直接拒绝，从源头杜绝了参数乱传的问题。

第二部分：AI 调用格式（Function Call），大模型给 Agent 的「标准化执行指令」

当大模型判断需要调用工具时，会严格按照固定格式返回 JSON，不再有任何自由发挥的自然语言。我们的 Agent 直接解析这个结构，拿到工具名和参数，不用做任何额外的猜解处理。

```
{
  "function_call": {
    "name": "check_weather",
    "parameters": {
      "city": "上海",
      "date": "2026-03-19"
    }
  }
}
```

注意 `function_call` 这个字段，它是一个固定标识，告诉 Agent：**这不是给用户看的回复，这是一条工具调用指令，你去执行它**。Agent 看到这个字段，就知道要去工具注册表里找 `check_weather` 这个函数，把 `city` 和 `date` 作为参数传进去。

你会发现，这里的 `date` 字段大模型自动把「明天」转换成了 `2026-03-19`，完全符合我们在工具定义里约定的 `YYYY-MM-DD` 格式。这就是标准化规范的威力，大模型知道格式要求，会自己做转换，不再把模糊的「明天」直接丢给工具。

第三部分：工具返回结果（Tool Response），Agent 把结果回传给大模型

Agent 拿到大模型的调用指令后，去执行对应的工具，然后把工具的执行结果，按规范格式回传给大模型。大模型会基于这个结果，决定下一步是继续调用其他工具，还是直接给用户返回最终答案。

```
{
  "city": "上海",
  "date": "2026-03-19",
  "temperature": "16-22℃",
  "condition": "多云转晴",
  "wind": "3级西北风"
}
```

这一步是整个循环的闭环，大模型不是一次性就完成任务的，它需要看到工具的执行结果之后，才能做出下一步决策。比如天气查到了，它会把这个结果整理成自然语言回复用户；如果工具报错了，它会决定换个方式或者告知用户。

对应 Agent 执行流程

把这三部分放回之前讲过的 Agent 执行循环，对应关系就清晰了。

先回忆一下 tools 文章里的那 7 步执行流程：

```
第 1 步：Agent 把「用户需求 + 工具清单」打包，发给大模型
第 2 步：大模型做决策，返回调用指令
第 3 步：Agent 执行指令，调用工具函数
第 4 步：Agent 把工具执行结果回传给大模型
第 5 步：大模型根据结果，决定下一步
第 6 步：循环执行，直到任务完成
第 7 步：Agent 把最终结果反馈给用户
```

Function Calling 规范的是其中有「数据交换」的三步：

第 1 步 → 工具定义（Tool Definition）

Agent 要把工具清单传给大模型，这份清单就是用 Function Calling 规定的 JSON 格式写的，每个工具都有标准化的 name、description、parameters 字段。大模型收到的不是一段随意的文字描述，而是严格规范的结构化数据。

第 2 步 → AI 调用格式（Function Call）

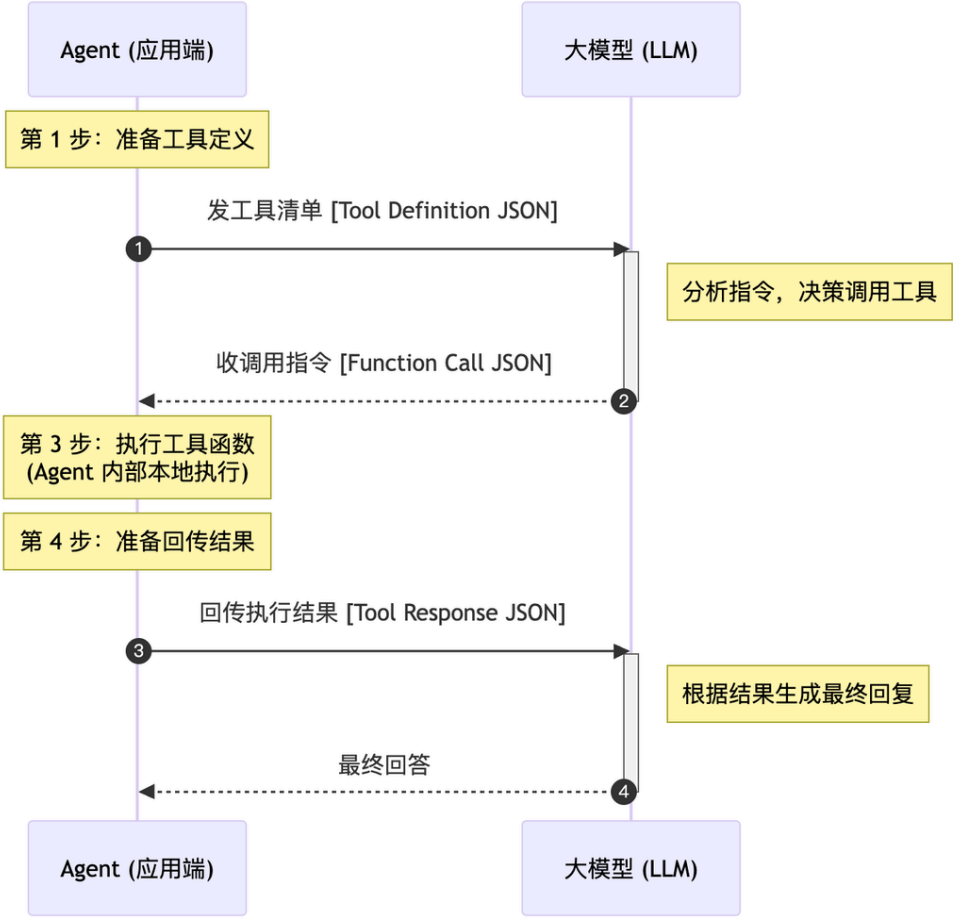
大模型做出「调用工具」的决策后，它返回给 Agent 的不是一句自然语言（比如「你去帮我查一下天气」），而是严格的 JSON 格式指令，`function_call.name` 告诉 Agent 调哪个工具，`function_call.parameters` 告诉 Agent 传什么参数。Agent 直接解析，不用猜。

第 4 步 → 工具返回结果（Tool Response）

Agent 执行完工具（第 3 步），拿到结果，按规范格式打包回传给大模型。大模型收到这个结果，才能做出下一步决策，是继续调工具，还是直接回答用户。

第 3 步不在 Function Calling 的管辖范围内，那一步是 Agent 实际执行函数，是真正「干活」的部分，不涉及大模型和 Agent 之间的通信格式。

用一张图把对应关系标出来：



Function Calling 把第 1、2、4 步的数据格式全部标准化了，这三步恰好是大模型和 Agent 之间所有的「信息交换点」。有了这套规范，双方说的是同一种语言，不再靠猜。

1. **解析工具名**：从 `function_call.name` 拿到 `"check_weather"`
2. **查找对应函数**：在工具注册表里，找到 `check_weather` 对应的实际函数
3. **传入参数执行**：把 `function_call.parameters` 里的 `city` 和 `date` 原样传进函数
4. **拿到结果回传**：函数执行完毕，把返回的天气数据回传给大模型

整个过程干净利落，不需要任何猜测和解析，因为 Function Calling 已经保证了调用指令的格式是固定的、可预期的。Agent 只需要按格式拆开、对号入座、执行、回传，就完成了自己在这一轮循环中的全部工作。

一张表看懂：System Prompt 传统模式 vs Function Calling 标准化模式

对比维度	System Prompt 传统模式	Function Calling 标准化模式
工具描述方式	自然语言随意写，无统一规范	固定 JSON 格式，强制规范 name/description/parameters 必填项
调用返回格式	完全靠 AI 自由发挥，五花八门，无固定结构	严格固定 JSON 结构，字段统一，可直接解析
参数错误率	极高，经常出现参数顺序错、格式错、模糊值	极低，大模型严格按参数规范生成，格式和必填项有服务端校验
开发成本	极高，需要写大量解析、纠错、重试逻辑	极低，直接解析固定格式，无需处理额外兼容问题
模型兼容性	极差，换个模型就要重写一整套提示词	极好，主流大模型通用，一套工具定义可跨模型使用

总结

整理一下这一章的核心认知：

- **Function Calling 是什么**：大模型和 Agent 之间的标准化工具调用协议，解决的是「大模型做出调用决策后，如何精准传递指令」的问题，是整个 Agent 开发的底层基础。
- **为什么需要它**：没有 Function Calling 之前，只能用自然语言在 System Prompt 里描述工具，大模型的回复格式完全不可控，开发者要写大量解析和纠错代码；

有了 Function Calling，双方都遵循固定的 JSON 规范，彻底告别猜谜。

- **三部分结构**：工具定义（我能用什么）→ AI 调用格式（我要调哪个）→ 工具返回结果（调完得到什么），对应 Agent 执行循环的第 1、2、4 步。

后续章节呼应：

- **RAG**：一种特殊工具，专门解决「大模型看不到你私有数据」的问题，重要到单独一章
- **MCP**：工具的标准化协议，不用每次都自己写工具定义，直接接入现成生态